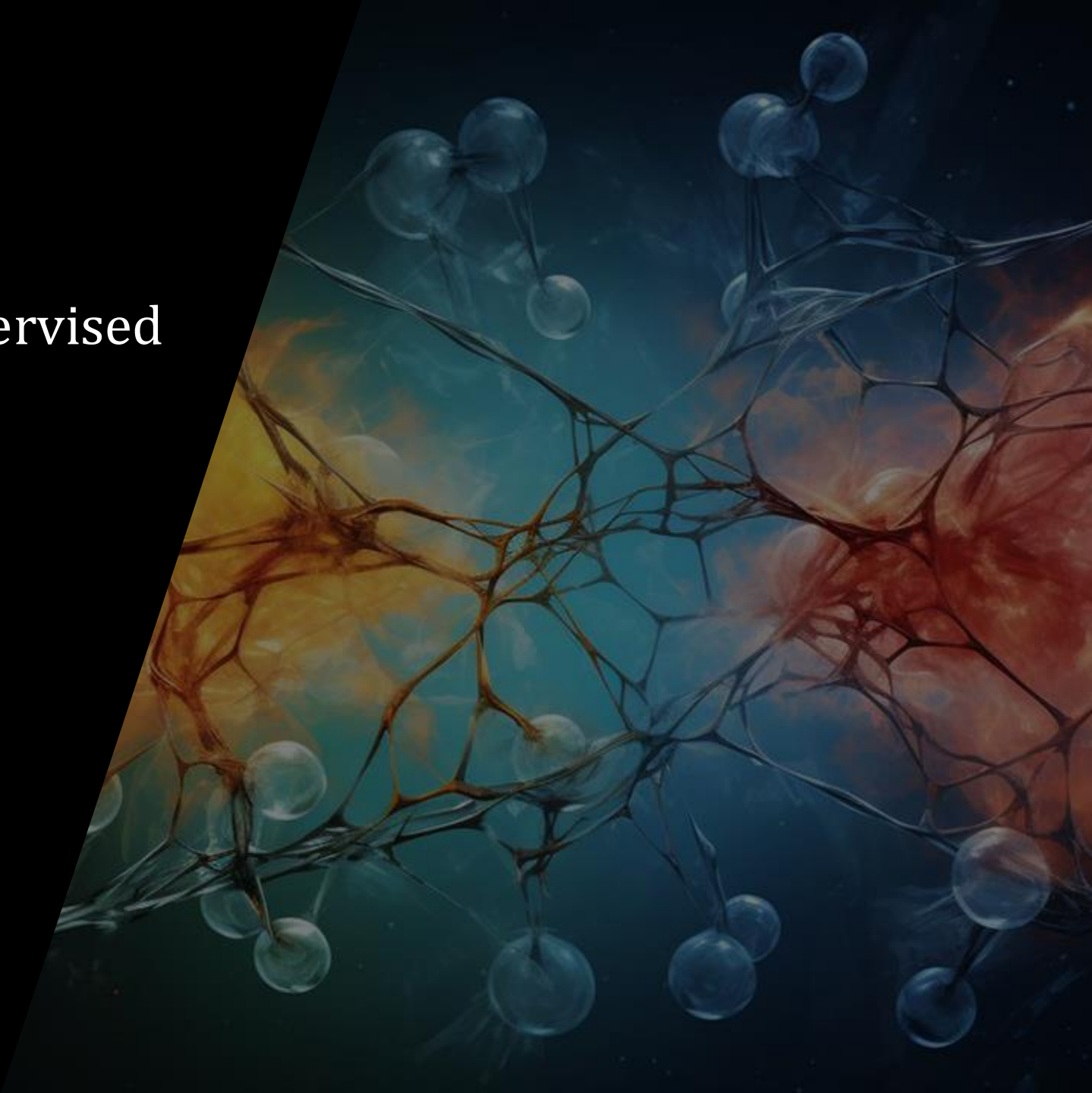


Session 5.1 – Principles of Supervised Learning

Contents:

- *Bias-variance trade-off for supervised problems*
- *Overfitting, underfitting and test set*
- *Implicit and explicit regularisations*
- *Case-study: Ridge Regression*



In the previous chapter, we have:

1. Specified different models for different supervised learning tasks (regression and classification),
2. Specified loss functions and associated empirical risks,
3. Used a finite training set to minimise the empirical risk,
4. Found parameters of our models $f_{\theta}(\phi^{(i)})$.

Now what could possibly go wrong with our models?

Generalisation!

Is it able to work on new, independent from training, data?

- The whole aim of training supervised models is to generalise well on unseen data. In practice, we would like to minimise $\mathbb{E}_{x,y}(\ell(f_{\theta}(x), y))$ that we approximate by $R_{\text{train}} = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x^{(i)}), y^{(i)})$
- In a regression context, suppose there exists f such that $y^{(i)} = f(x^{(i)}) + \epsilon^{(i)}$, with $\mathbb{E}[\epsilon^{(i)}] = 0$, $\mathbb{E}[\epsilon^{(i)^2}] = \sigma_{\epsilon}^2$
- We build a model f_{θ} of f minimising the squared error $\ell(f_{\theta}(x^{(i)}), y^{(i)}) = (y^{(i)} - f_{\theta}(x^{(i)}))^2$
- We can show that the expected risk on a **test example** $\tilde{x}$ (never seen by the model during training) decomposes as

$$\mathbb{E}[(y - f_{\theta}(\tilde{x}))^2] = \mathbb{E}[f_{\theta}(\tilde{x}) - f(\tilde{x})]^2 + \mathbb{E}[(f_{\theta}(\tilde{x}) - \mathbb{E}[f_{\theta}(\tilde{x})])^2] + \sigma_{\epsilon}^2$$

$$\mathbb{E}[(y - f_{\theta}(\tilde{x}))^2] = \text{Bias}[f_{\theta}(\tilde{x})]^2 + \text{Var}[f_{\theta}(\tilde{x})] + \sigma_{\epsilon}^2$$

Note: Every terms are >0

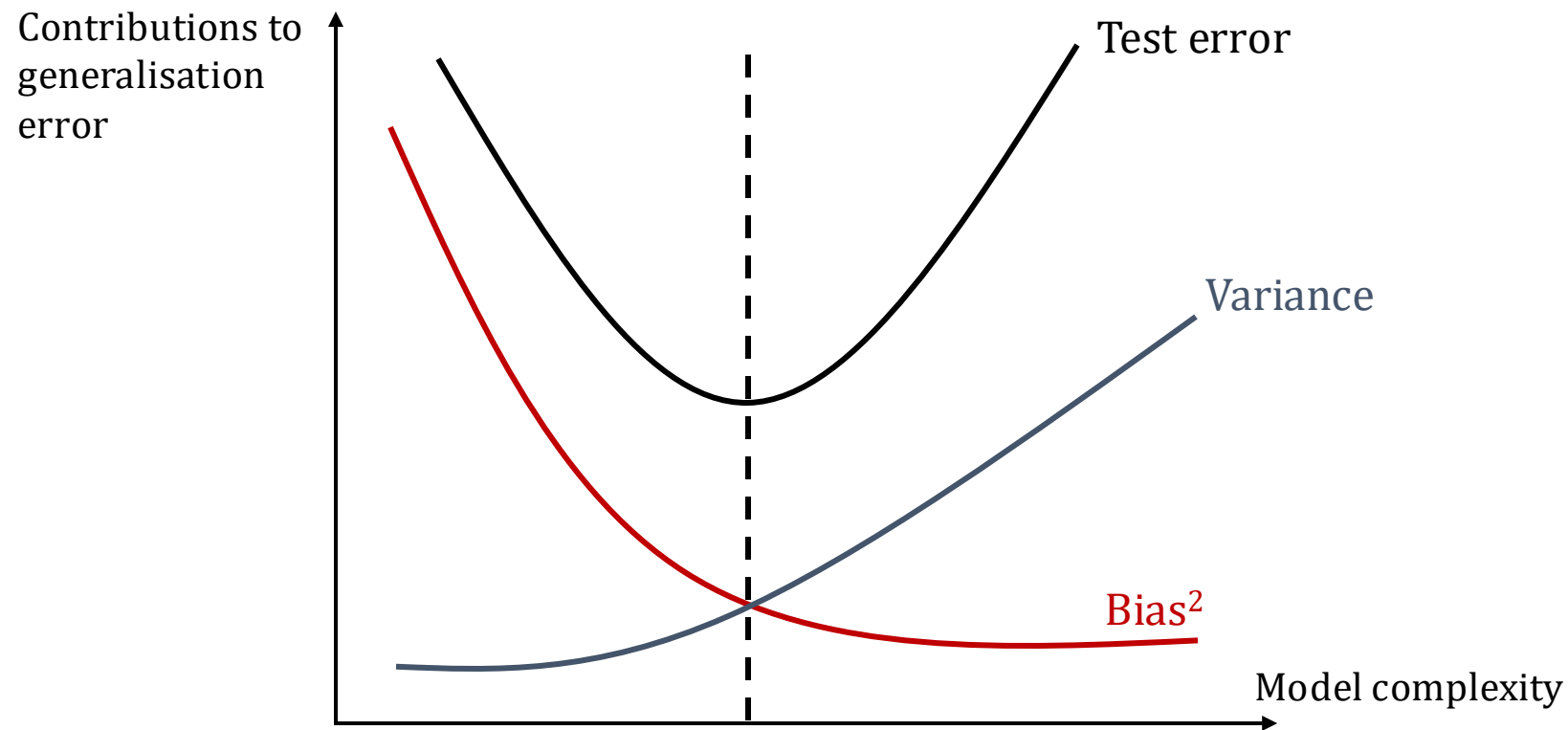
↓
Modelling
error

↓
Model
variability

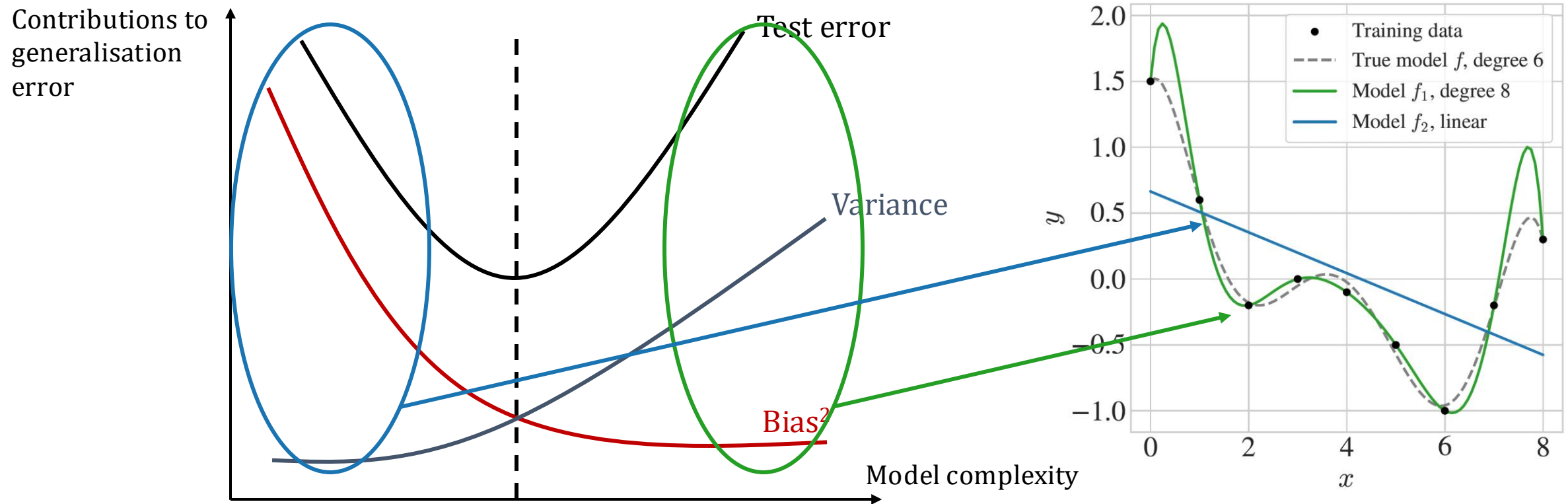
↓
Irreducible
error

Note: a similar expression holds for classification

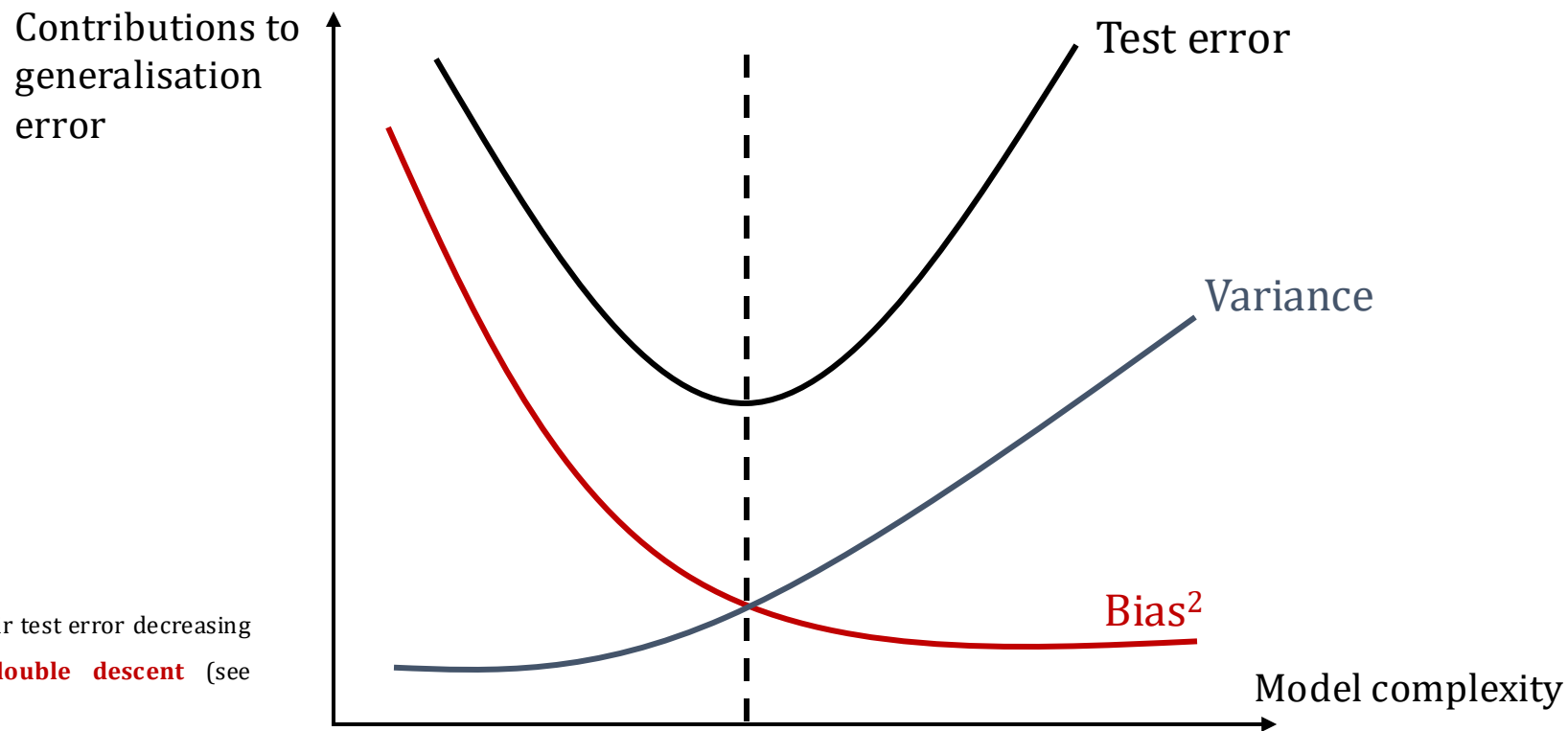
- **“Simple” models** have **large bias** because they constrain very much the function class that is therefore far from the truth, but they usually have **low variance** and are robust to variations of the training set
- **“Complex” models** (with a lot of parameters for instance) have **small bias** but **large variance**



- **“Simple” models** have **large bias** because they constrain very much the function class that is therefore far from the truth, but they usually have **low variance** and are robust to variations of the training set
- **“Complex” models** (with a lot of parameters for instance) have **small bias** but **large variance**



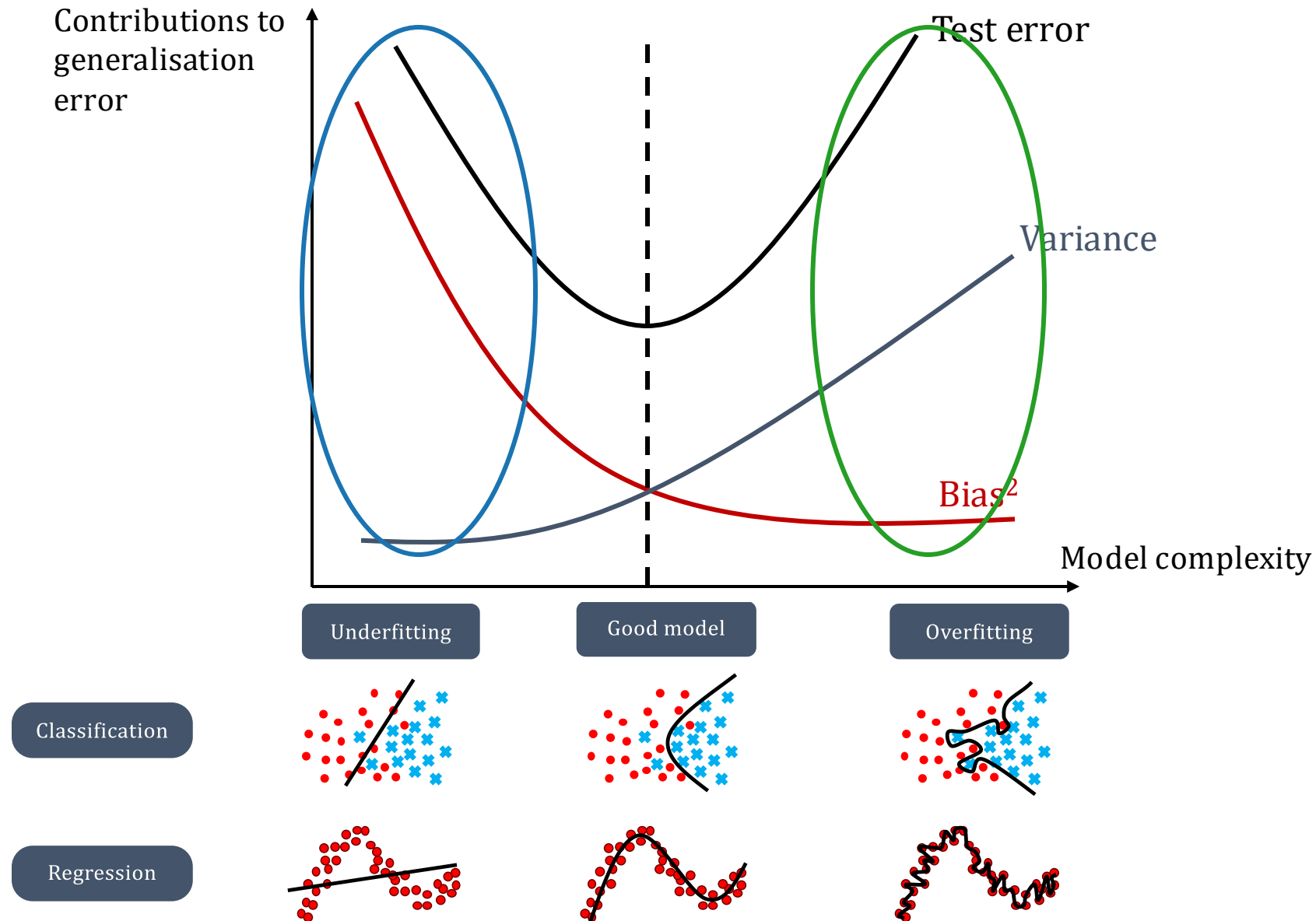
- “**Simple**” models have **large bias** because they constrain very much the function class that is therefore far from the truth, but they usually have **low variance** and are robust to variations of the training set
- “**Complex**” models (with a lot of parameters for instance) have **small bias** but **large variance***



* Heavily overparametrized have their test error decreasing again, a phenomenon dubbed **double descent** (see [Belkin+18](#) and [Nakkiran+19](#))



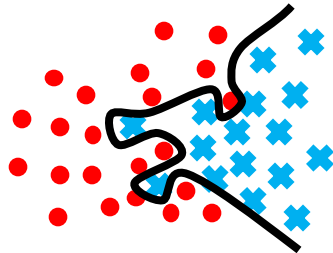
Models need to be built such that they are not too flexible to fit the noise in the data but also not too restrictive to avoid bias



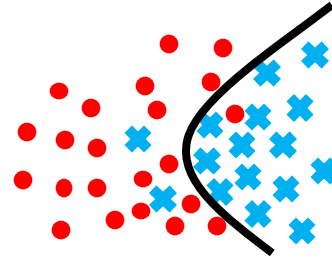
- To measure if we really learnt something useful in supervised learning in practice, we use a **test dataset** that the model has never seen but for which we know the labels and check that $R_{\text{train}}(\hat{\theta})$ and $R_{\text{test}}(\hat{\theta})$ are of the same order
- Based on the previous view, there are two regimes where things could go wrong: high variance and low bias models vs high bias and low variance models, respectively defining **overfitting** and **underfitting**

Classification

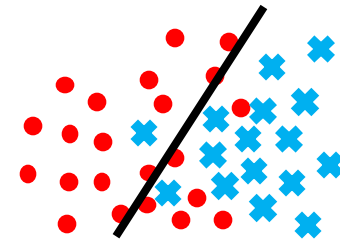
Overfitting



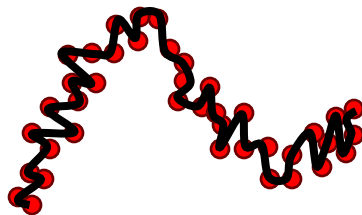
Good model



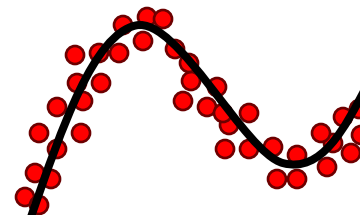
Underfitting



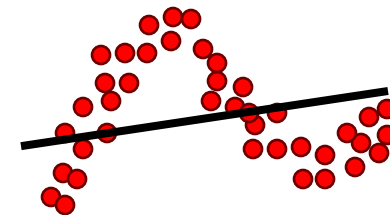
Regression



$$R_{\text{train}}(\hat{\theta}) \ll R_{\text{test}}(\hat{\theta})$$



$$R_{\text{train}}(\hat{\theta}) \approx R_{\text{test}}(\hat{\theta})$$

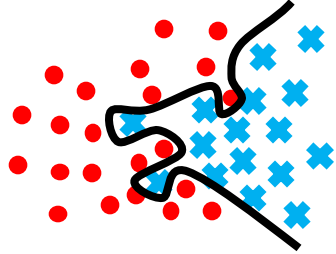


$$R_{\text{train}}(\hat{\theta}) \approx R_{\text{test}}(\hat{\theta})$$

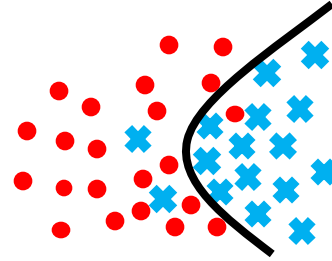
but large

Classification

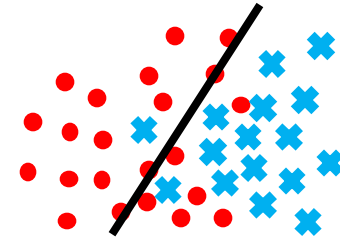
Overfitting



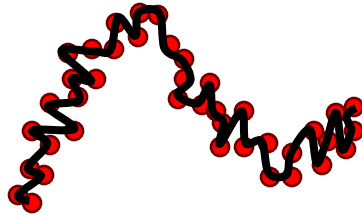
Good model



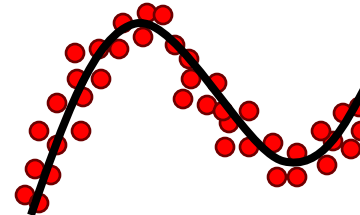
Underfitting



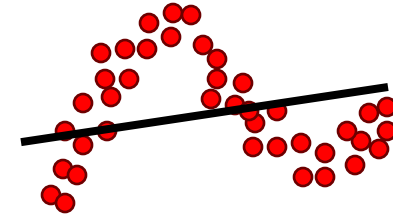
Regression



$$R_{\text{train}}(\hat{\theta}) \ll R_{\text{test}}(\hat{\theta})$$



$$R_{\text{train}}(\hat{\theta}) \approx R_{\text{test}}(\hat{\theta})$$



$$R_{\text{train}}(\hat{\theta}) \approx R_{\text{test}}(\hat{\theta})$$

but large

Possible fixes

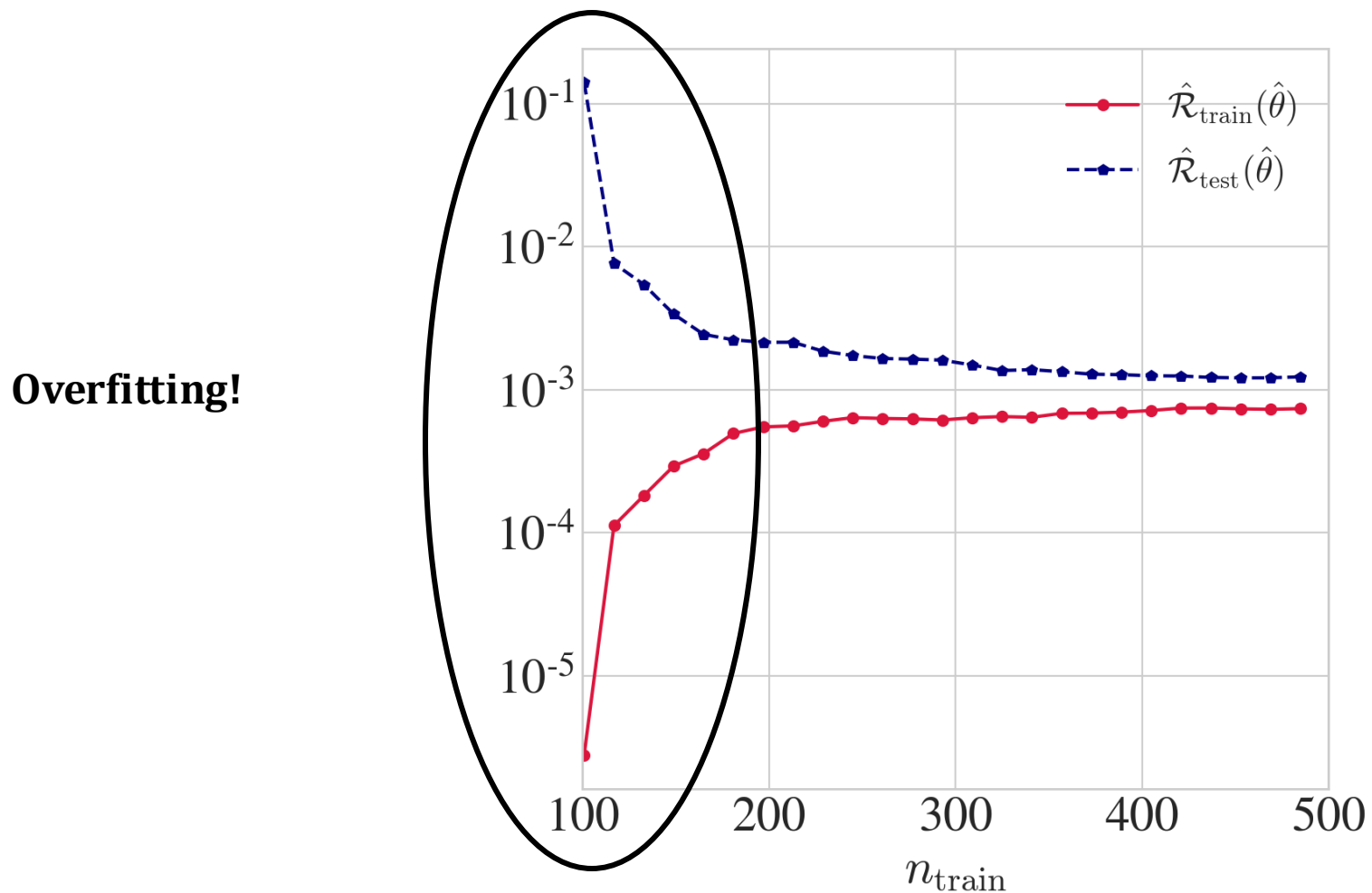
- Add more data
- Remove features
- Stop the training earlier
- Add **regularisation**

- You did a good job!

- Try a more complex model
- Train your model longer

- Back to our regression problem: what happens when $n \approx d'$? Remember the solution

$$\hat{\theta} = (\Phi^T \Phi)^{-1} \Phi^T y$$



- One generic way to deal with **overfitting** is by penalising ‘complex’ models and restrain the class of learned functions: this is called explicit **regularization** and appears in the optimization problem as a constraint on parameter space

$$\hat{\theta} = \underset{\theta \in \Theta}{\operatorname{argmin}} R(\theta)$$

Unregularized
optimization

$$\hat{\theta} = \underset{\theta \in \Theta}{\operatorname{argmin}} R(\theta) \text{ s.t. } P(\theta) \leq \epsilon$$

Regularized
optimization

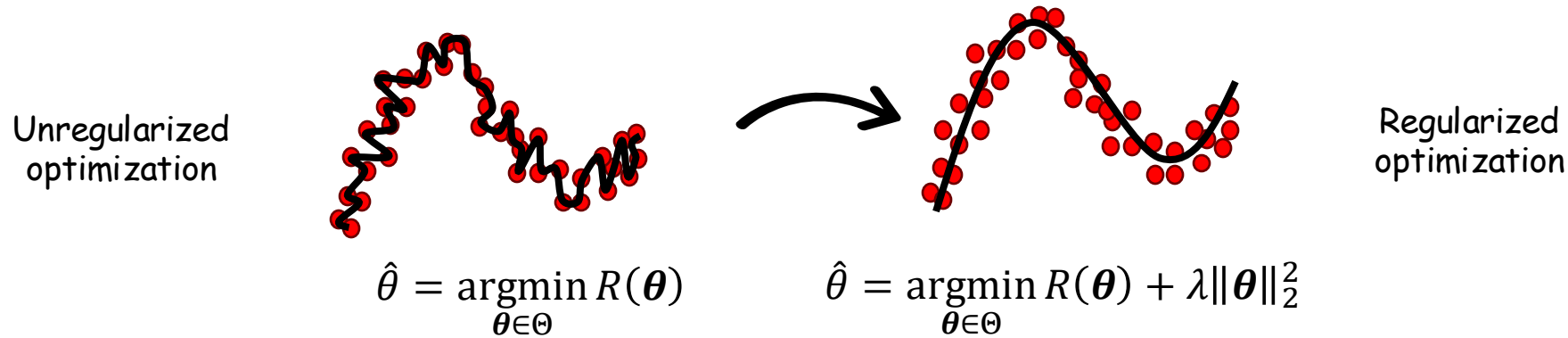
- The function $P(\theta)$ is called **penalty function** and **imposes a constraint on the form of the solutions**. The regularized problem can be written equivalently as (by Lagrange duality)

$$\hat{\theta} = \underset{\theta \in \Theta}{\operatorname{argmin}} R(\theta) + \lambda P(\theta)$$

- λ is a **regularization (hyper)parameter**
 - $P(\theta)$ can take different forms depending on the penalty we chose to impose on the set of parameters
- Common penalty functions are the L_p -norms with $p = 1$ or 2

$$P(\theta) = \|\theta\|_p$$

- By solving the regularized optimization with **an appropriate value of λ** , we can reduce the overfitting



- Let us apply L_2 regularisation to our linear regression model which now minimises the regularised risk

$$\hat{\theta} = \operatorname{argmin}_{\theta \in \Theta} \frac{1}{n} \|\Phi \theta - Y\|_2^2 + \lambda \|\theta\|_2^2$$

- This can be minimised analytically and gives the **Ridge Regression Model**

$$\hat{\theta} = \frac{1}{n} \left(\frac{\Phi^T \Phi}{n} + \lambda I \right)^{-1} \Phi^T Y$$

where the matrix is now invertible for $\lambda > 0$.



The penalty impact the generalisation error by increasing the bias and decreasing the variance with λ

→ **Regularisation helps reducing overfitting and improves the generalisation performances!**

